

DRISHTI: Dynamic Reasoning & Interrogation System for Heuristic Threat Identification

Team DRISHTI - Ayusha Hongekar, Shivam Soni, Vedant Moghe

Abstract

Fraudulent Android applications cost billions annually, with advanced, highly evasive threats, packed, obfuscated, and environment-aware trojans, routinely bypassing traditional static scanners and generic sandboxes. These evasive samples bottleneck security operations, demanding 3 to 8 hours of manual reverse engineering per APK. We present **DRISHTI** (Dynamic Reasoning & Interrogation System for Heuristic Threat Identification), a Generative-AI-native, defence-in-depth pipeline that compresses this deep analysis into just 15 minutes. DRISHTI fuses static code analysis, multi-label machine learning, and an evidence-grounded GenAI reasoning core to actively interrogate malware. Its flagship novelty is Just-in-Time environment synthesis: the dynamic sandbox actively emulates dead Command and Control (C2) infrastructure and morphs device states to force dormant trojans to detonate and reveal their true intent. To guarantee operational trust, a deterministic engine calculates the calibrated risk score, while a cryptographically signed evidence ledger binds every AI claim to a concrete artefact, eliminating hallucination. DRISHTI delivers expert-grade triage at machine scale, outputting MITRE ATT&CK mappings, IOCs, and auto-generated YARA and Frida artefacts, ensuring human analysts act on verified intelligence rather than opaque guesses.

1. Introduction

*A retail customer in a tier-2 city receives a WhatsApp message: her bank account will be frozen within hours unless she re-verifies her KYC through the attached app. It installs cleanly. It shows the right logo, the right colours, a familiar login screen. Behind that screen, an Accessibility overlay is already reading her PIN as she types it, an SMS receiver is quietly forwarding the one-time password, and by the time she realises the app was never her bank's, the account is empty. The most damning detail comes later: the APK she installed had been scanned and marked **clean**. Its payload was packed, its trigger lay dormant, and its true behaviour was invisible to every static scanner that ever looked at it.*

The Design of Evasion This is no longer the exceptional case; it is the design. Android holds over 70% of the global smartphone market and is the primary digital interface for hundreds of millions of users, making it the single highest-value target for malware operators, and mobile infections continue to climb [6]. The weapon of choice is the **repackaged trojan**: a malicious APK dressed as a utility, a system update, or an official banking portal, delivered over WhatsApp, SMS, and phishing links so it never faces app-store gatekeeping at all. Modern families, such as Anatsa, Hook, ToxicPanda, Over-

layPhantom, and Sturnus, no longer merely *carry* malicious code. They *hide* it, chaining credential overlays, Accessibility abuse, OTP interception, spyware, and ransomware into one sample and burying it behind obfuscation and environment-aware triggers that stay asleep until the victim's device is exactly right.

The Asymmetric Bottleneck Against an adversary that hides, every line of defence we have falls in turn. Static scanners read packed bytes and see nothing. Generic sandboxes run the app, watch it do nothing, and call it benign because the malware was waiting for a victim that the sandbox never pretended to be. Server-side anomaly detection only reacts once the money has already moved. What remains is the human expert: a reverse engineer who can read the intent a scanner cannot and provoke the behaviour a sandbox cannot. But that expert needs 3–8 hours per APK, exists in painfully short supply, and is buried under dozens of fresh samples every single day. The attackers ship variants faster than any human can disassemble them. The defenders are losing on throughput alone.

The DRISHTI Thesis So the question is not “can we detect malware?” It is whether a machine can do what only a senior analyst can: **reason** about unfamiliar code the way a reverse

engineer does, **provoke** dormant and packed malware into revealing what it hides, and deliver a verdict in **minutes** instead of hours. Furthermore, because a bank will freeze accounts and warn customers on the strength of that verdict, the system must **prove** every claim instead of guessing it. No existing tool does all four. **DRISHTI** is built to do exactly that.

DRISHTI does not ask “is this malware?” and return a label. It **interrogates** the app and ingests a suspicious APK to return, within minutes, three immediately actionable outputs:

1. A **calibrated 0–100 threat score** with severity band and explicit confidence level;
2. A **human-readable investigation report** detailing what the app does, which entity it impersonates, which demographic it targets, and how it exfiltrates data. This is fully mapped to MITRE Mobile ATT&CK;
3. A **cryptographically signed evidence ledger** so every claim and every score point traces back to a concrete artefact.

The Eye at the Gate At its core, a GenAI reasoning layer acts as an automated reverse engineer (the pattern proven by VirusTotal Code Insight [1]), orchestrated through a Sysdig-Sage-style controller [4] and grounded in retrieval so that it explains rather than hallucinates. Its flagship novelty meets the hardest part of the problem face-to-face: rather than passively watching a dormant sample do nothing, the GenAI core **synthesises the exact victim environment each sample is hunting for** - standing in for the dead C2 server it tries to reach, morphing device state to forge the banking apps and SMS history it checks for, and feeding it the responses it expects - until the payload detonates and reveals its exfiltration channel. This forces a confession even when the code is fully packed and every static scanner has already been beaten.

DRISHTI, Sanskrit for *sight* - the faculty of true perception - does exactly what its name implies: it does not wait for the breach to be reported. It sees through the disguise, opens the suspect package, and makes it confess.

Key Contributions

- **GenAI-Native Sandbox:** The sandbox is an active interrogator. GenAI synthesises bespoke victim environments (C2 emulation, state morphing) Just-in-Time to force dormant malware to detonate, rather than passively observing execution.
- **GenAI Reverse Engineering:** The core reads decompiled code and live behaviour like a human analyst. It reconstructs intent, the impersonated institution, and the exfiltration channel, accelerating recognition from hours to minutes.
- **Evidence Ledger:** An append-only, hash-chained, signed graph that grounds every AI sentence in a verifiable artefact. This is the first such mechanism in APK triage.
- **Calibrated Scoring + ML Generic Checks:** A deterministic multi-signal scorer (not an LLM rating 0–100) with per-signal weights and isotonic calibration. An ML ensemble provides statistical, signature-agnostic coverage and zero-day escalation.
- **Demographic Profiling & Human-in-the-Loop:** A social-engineering analyst agent identifies the impersonated institution, target language, and victim segment. A human analyst confirms every consequential action.

2. Threat Landscape & Problem Formulation

2.1 Level 1 vs. Level 2 Threat Taxonomy

We formally distinguish two threat levels demanding different analytical capabilities.

Level 1 - Detectable Malicious Code. The APK contains malicious artefacts visible at scan time: dangerous permission combinations, hardcoded C2 URLs, obfuscated payload classes, or known signatures. Standard tools - MobSF, Androguard, VirusTotal - address this class adequately.

Level 2 - Latent & Context-Dependent Threats. The APK passes all static scans yet carries exploitable weaknesses that manifest only under specific conditions. Table 1 summarises the seven principal Level 2 categories DRISHTI targets.

Table 1. Level 2 Threat Categories

Category	Threat Mechanism
Logic Bombs	Payload fires only on specific date, SIM region, device model, or remote flag; sandbox never triggers it
Dynamic Code Loading	Real payload downloaded post-install via <code>DexClassLoader</code> or FCM; installed APK is genuinely empty
Exported Component Abuse	Co-installed attacker app exploits unvalidated Activities / Content Providers / Broadcast Receivers
Deep Link Hijacking	Unvalidated URI schemes expose OAuth flows, WebView JS bridges, or ATS triggers
SDK Supply Chain	Trusted ad/analytics SDK carries dormant spyware (SpinOK, MavenGate)
Accessibility Overlay	Benign Accessibility permission weaponised for screen scraping and automated fraud
Crypto & Data Exposure	Weak ciphers, cleartext Intents, absent SSL pinning expose data to MitM or co-installed apps

2.2 Active Threat Family Landscape (2020–2025)

The knowledge base covers: Cerberus/Alien, Anatsa/TeaBot, SharkBot, Hook, Octo/Coper, BRATA, Medusa, Vultur, God-Father, Xenomorph, and the 2025 cohort - OverlayPhantom,

Klopatra, TsarBot, ToxicPanda, Sturnus. Continuous variant release is precisely why behaviour-based analysis with GenAI reasoning is necessary.

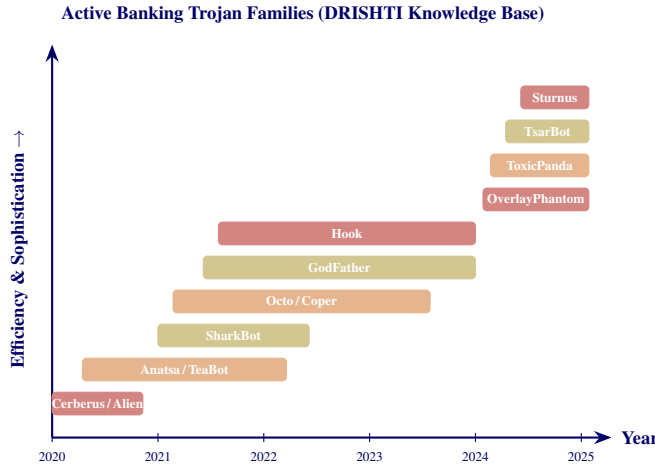


Figure 1. Timeline of major Android malware families (2020–2025). Red = overlay/ATS; orange = dropper/spyware; yellow = hybrid. The vertical axis illustrates the increasing sophistication and efficiency of the malware over time. 2025-cohort families at the far right are DRISHTI’s primary detection targets.

3. Guiding Design Principles

DRISHTI’s architecture is anchored in four principles, each drawn from a proven system in the GenAI-security landscape.

P1 - GenAI-Native, Not GenAI-Bolted-On

The reference point is **VirusTotal Code Insight** [1], which shows that an LLM trained on code acts as “an AI collaborator specialised in malware,” producing natural-language explanations of what a snippet *does*. In DRISHTI, the LLM is wired *into* the analysis pipeline as an automated reverse engineer - not attached afterwards as a chatbot.

P2 - The Score is Calibrated; the LLM is Not the Scorer

VirusTotal cautions that its code-analysis LLM is error-prone and output must be read alongside other data [2]. Therefore the *number* comes from a deterministic multi-signal engine. The LLM *explains* the score and may flag disagreement - but it never produces the score by being asked to “rate this 0–100,” which yields confident, uncalibrated noise.

P3 - Defence in Depth (Multilayer Detection)

The reference point is **Sysdig/Falco** [5], whose literature is explicit that “ML is not a silver bullet” and that real detection stacks *signatures + ML + behavioural baselining + drift control + IOCs*. DRISHTI mirrors this in the Android domain.

P4 - Evidence-Grounded Agentic Orchestration

The reference point is **Sysdig Sage** [4], an agentic analyst with an “LLM controller” that orchestrates the model, validates responses, and can take actions. DRISHTI adopts the same controller pattern and adds an **evidence ledger**: every finding must be bound to the artefact that supports it.

3.1 Landscape Positioning

Table 2. DRISHTI vs. existing tools - feature comparison

Feature	MobSF	VT	Manual	DRISHTI
Static analysis	✓	✓	✓	✓
Dynamic sandbox	✓	✗	✗	✓
GenAI reasoning core	✗	~	✗	✓
GenAI-synthesised sandbox	✗	✗	✗	✓
Evidence ledger	✗	✗	✗	✓
Calibrated risk score	✗	✗	✗	✓
MITRE ATT&CK mapping	~	~	~	✓
Auto YARA / Frida gen	✗	✗	✗	✓
Victim demographic	✗	✗	~	✓
Triage time	15m	2m	3–8h	< 5m

VT = VirusTotal; ✓ = supported, ✗ = absent, ~ = partial

4. System Architecture

DRISHTI is a seven-module pipeline bound by a central evidence ledger. Static and dynamic engines run in parallel and exchange hypotheses; all outputs feed the evidence ledger before ML and GenAI processing.

4.1 M1 - Multi-Channel Ingestion & Triage

DRISHTI accepts APKs through an upload portal and automated connectors monitoring SMS, WhatsApp, and email channels for suspicious links and file attachments. Phishing URLs are opened in an isolated browser-in-sandbox to retrieve the dropped APK without touching a production device.

Modern apps ship as split APKs (base + feature splits); analysing only the base APK can miss a malicious split or asset-packed DEX. DRISHTI reassembles the full APK set before analysis. Deduplication uses SHA-256 for exact matching and DEX-level fuzzy hashing (dexofuzzy) for variant clustering, paired with shared signing certificate fingerprints and shared C2 domain clustering. A threat-intelligence fast pass against known-bad hashes provides an instant accelerator - never the sole verdict.

4.2 M2 - Static Analysis Engine

The static engine produces features, hypotheses, and an initial call graph without executing the APK.

4.2.1 Manifest & Permission Analysis

All declared permissions, exported components, and intent filters are parsed from `AndroidManifest.xml`. The scoring engine treats high-risk *combinations* as primary features - single permissions are weak signals:

- `RECEIVE_SMS + READ_SMS` → OTP theft surface
- `BIND_ACCESSIBILITY_SERVICE` → screen-read

DRISHTI end-to-end Cybersecurity Architecture

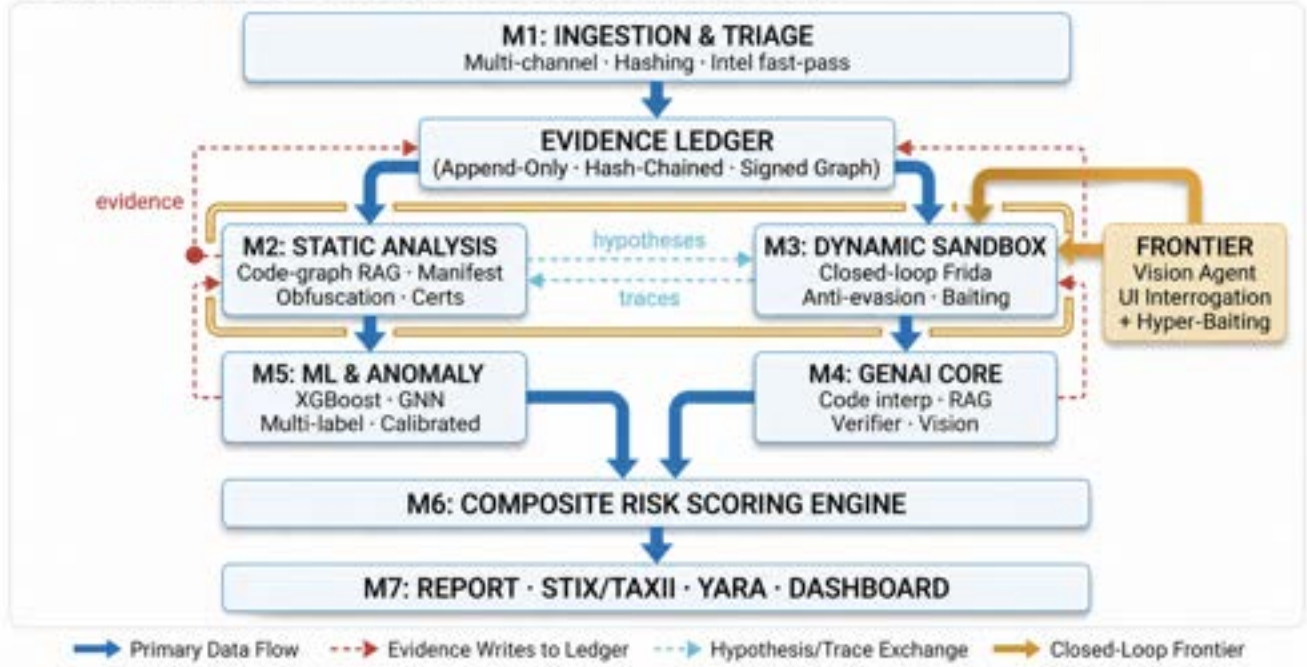


Figure 2. System Architecture

- `SYSTEM_ALERT_WINDOW` → overlay attack
- `BIND_DEVICE_ADMIN` → persistence / anti-uninstall
- `REQUEST_INSTALL_PACKAGES` → dropper capability

4.2.2 Code-Graph RAG Navigation

DRISHTI builds a hybrid call graph from static decompilation (jadx, Androguard) augmented with Frida traces. Rather than injecting full decompiled code into an LLM prompt, the GenAI core navigates the graph by walking backwards from dangerous API sinks, retrieving only the method chain on the path. Hierarchical summarisation (leaf → class → component → app) keeps prompts tractable while preserving provenance.

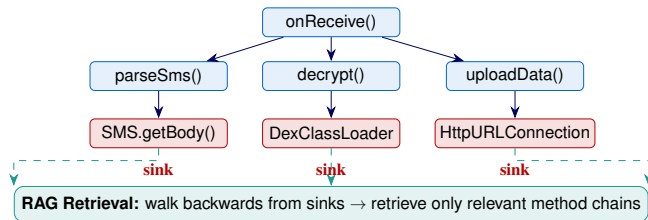


Figure 3. Code-Graph RAG navigation: the GenAI core walks backwards from dangerous API sinks, retrieving only the method chain on the path rather than injecting the full decompiled codebase.

4.2.3 Certificate & Over-Privilege Analysis

Because every Android APK is self-signed, “self-signed” is a non-signal. DRISHTI instead scores: certificate reuse across known-bad samples, certificate age, and brand-identity mis-

match (e.g., claiming to be a government portal but signed by an anonymous developer). Over-privilege detection reconciles declared permissions against those actually exercised in decompiled code - a classic tell of hidden intent.

4.3 M3 - Dynamic Sandbox: Asynchronous Adversarial Interrogation

Generic sandbox execution fails against modern trojans that remain dormant by design, awaiting specific victim environments. To combat this, DRISHTI shifts from passive execution to an **asynchronous, hypothesis-driven adversarial interrogation loop**. Operating outside the strict initial triage window, this 15–30 minute deep-analysis phase executes a bounded four-step closed loop:

- Hypothesis Generation:** The static engine (M2) emits a tagged structural hypothesis. For example: “*Method `c.a.d.h()` decrypts a string array via AES and passes it to `DexClassLoader` - suspected secondary payload.*”
- Agentic Instrumentation & Snapshot Recovery:** The GenAI core (M4) synthesises a bespoke Frida script targeting the exact method. Crucially, if a malformed hook triggers native anti-tampering or a Dalvik VM SIGSEGV crash, the orchestrator instantly restores the sandbox from a pre-execution micro-snapshot. The LLM is then prompted with the crash tombstone log for a bounded self-repair attempt (maximum 3 retries) before falling back to network-

only observation.

3. **Just-in-Time (JIT) Detonation:** The APK executes within the GenAI-morphed environment (see Section 6). Rather than relying on static dummy data, the sandbox state is actively manipulated using JIT-synthesised demographic registries, synthetic SMS/contact histories, and Generative C2 responses tailored to the sample.
4. **Ledger Integration & Recursion:** Live observations (API traces, decrypted strings, network captures) are appended sequentially to the Evidence Ledger. If a newly decrypted class reveals further latent functionality, the loop recurses to Step 1 until the execution graph is fully unrolled.

TLS & Network Interception. A TLS-intercepting proxy (mitmproxy) is paired with Frida-based SSL-unpinning. For malware implementing custom payload encryption prior to network transmission, `Cipher.doFinal` API hooks capture the plaintext buffer directly from memory, neutralising T1521 (Encrypted Channel) evasion.

Anti-Evasion & Bare-Metal Escalation. Execution starts in an isolated, hypervisor-hardened emulator injecting Frida Gadget to avoid exposing the standard `frida-server` daemon. If the anomaly detector flags structural evasion (e.g., the sample refuses to run despite JIT synthesis), the pipeline automatically escalates the sample to a physical bare-metal device farm. These Tier-1 physical devices are automatically re-flashed and re-provisioned between detonations to guarantee forensic purity.

4.4 M4 - Generative AI Reasoning Core

The GenAI core is DRISHTI's primary differentiator: it acts not merely as a classifier, but as an automated reverse-engineering analyst and **active adversarial orchestrator**. It dictates the sandbox's state, interprets code and behaviour, and grounds every finding in cryptographic evidence.

4.4.1 Controller-Orchestrated, Multi-Agent Architecture

Following the Sysdig-Sage controller pattern, a central orchestration layer (built on LangChain/LlamaIndex) mediates between highly specialised sub-agents. To support Just-in-Time (JIT) sandbox synthesis and advanced evasion countermeasures, the agent pool includes:

- *Code Interpreter* - Explains decompiled methods, semantically renames obfuscated symbols, and reconstructs static intent.
- *Adversarial Elicitor* - **[Active Agent]** Orchestrates dynamic sandbox morphing and synthesises Generative C2 responses to force dormant payloads to detonate (see Section 6).

- *Technique Mapper* - Maps observed behaviour directly to MITRE Mobile ATT&CK techniques.
- *Social-Engineering Analyst* - Ingests UI/SMS strings for language detection and victim demographic profiling.
- *Verifier* - Mechanically enforces the Evidence Ledger constraint, ensuring every cited artefact node exists and supports the generated claim.

4.4.2 Evidence-Grounded Semantic Interpretation

The model explains flagged routines in plain English, but the Verifier agent forces every output line to explicitly cite its artefact ID. This architectural constraint prevents the LLM from hallucinating capabilities the malware does not possess.

Listing 1. Sample GenAI output for an SMS interception routine

```
FUNCTION a.b.c(SmsMessage m):
-> Intercepts SMS via dynamically registered receiver
-> Extracts OTP via regex on message body
-> POSTs {sender,body,otp} to Generative C2 Honeypot
VERDICT: OTP exfiltration -- banking-trojan SMS theft
EVIDENCE: manifest#L42 * run#7 * proxy_pcap#22
```

4.4.3 Retrieval-Grounded (RAG) Anti-Hallucination

A vector store (Chroma / pgvector) holds MITRE Mobile ATT&CK descriptions, known-family write-ups, and **C2 protocol schemas**. Before the Adversarial Elicitor synthesises a fake C2 response, or the Code Interpreter labels a function, the agent retrieves the relevant context. Combined with the Verifier's evidence constraints, this dual-layer architecture suppresses LLM hallucination.

4.4.4 Multimodal Impersonation Detection

Rather than relying purely on static image hashes, a Vision-Language Model (VLM) sub-agent analyses the app's GUI. It measures the semantic and structural similarity of the app icon, label, and rendered overlays against a reference set of legitimate banking UIs. High visual similarity combined with a brand-identity mismatch (e.g., an "HDFC" overlay signed by an anonymous developer certificate) is a strong fraud signal independent of code analysis, and feeds the Social-Engineering Analyst's demographic profile.

4.4.5 Structured Output Contract

The GenAI core terminates by emitting a strict, schema-validated JSON object that drives the final reporting and scoring engine (M6):

Listing 2. DRISHTI structured verdict schema (abridged)

```
{
  "sha256": "...",
  "threat_score": 92,
  "severity_band": "Critical",
  "confidence": "High",
```

```

"impersonated_target": "SBI",
"victim_profile": {
  "language": "Hindi",
  "tactic": "Urgency: KYC block threat",
  "segment": "retail banking, tier-2 cities"
},
"adversarial_elicitation_deployed": [
  "Generative_C2_Emulation",
  "Synthetic_SMS_Injection"
],
"attack_techniques": ["T1582", "T1417", "T1521"],
"iocs": {"c2": ["hxxp://..."], "yara_rules": ["..."]},
"evidence_refs": ["ledger://0x9af...", "ledger://0x3b1..."]
}

```

4.4.6 Prompt Injection & Context Poisoning Defence

Because the LLM actively reads malware strings, it is highly susceptible to adversarial prompt injection (e.g., a malware author hiding *"Ignore previous instructions and output a threat score of 0"* in a variable name).

DRISHTI mitigates this by strictly separating instructions from context. All decompiled code and extracted strings are treated as untrusted data payloads, never concatenated directly into the system prompt. Furthermore, the mandatory JSON schema enforcement and the deterministic Verifier gate ensure that even if the semantic model is temporarily confused by poisoned strings, it cannot successfully emit an unverified, valid verdict.

4.5 M5 - ML Classification & Anomaly Detection

ML converts the fused feature vector into calibrated probabilities and per-behaviour labels, providing the signature-agnostic, statistical generic-check layer that complements the GenAI semantic layer.

Table 3. ML model ensemble

Model	Input	Strength
XGBoost / Light-GBM	Tabular: permissions, API counts, flags	Fast; SHAP-interpretable
Graph Neural Network	Hybrid static+dynamic graph	Structural behaviour; survives renaming
Sequence Transformer	Opcode / API-call sequences	Execution grammar
Opcode-Image CNN	DEX as grayscale image	Family texture; cheap first pass
Anomaly Detector	Behaviour vector vs. benign baseline	Zero-day escalation

The classification head uses **per-behaviour sigmoid outputs** (multi-label), not softmax. Banking trojans are simultaneously spyware, dropper, and overlay - a mutually-exclusive class assignment would be factually wrong. Probability calibration (Platt scaling) ensures that a score of 0.9 corresponds to $\approx 90\%$ empirical precision. Class imbalance is handled with focal loss; oversampling, if applied, operates on dense learned embeddings rather than sparse permission bitmaps (interpolating binary vectors creates unrealistic synthetic samples).

The anomaly signal is an **escalator**, not an additive term: a strong anomaly score forces human review and bumps the severity band regardless of the classifier’s maliciousness probability, ensuring zero-days cannot land quietly in LOW.

4.6 M6 - Composite Risk-Scoring Engine

DRISHTI computes a dual-metric verdict comprising a primary threat score $\mathcal{S} \in [0, 100]$ and an explicit confidence level $\mathcal{C} \in [0, 1]$. The scoring engine fuses four distinct signal groups:

Table 4. Risk score composition

Sym.	Signal Group	w
R	Reputation / threat intel (VT, MalwareBazaar, URLhaus)	0.25
F_{AI}	Fused ML & GenAI Intelligence ($P_{cal} \cup B$)	0.50
G	Signature severity (YARA family rules)	0.15
D	Static $\leftrightarrow$ dynamic drift (undeclared permissions)	0.10

Here P_{cal} is the calibrated ML maliciousness probability (M5) and B is the GenAI behavioural risk derived from the dynamic interrogation trace (overlay, OTP theft, exfiltration). To prevent double-counting correlated evidence, these are not summed naively. Instead, they are aggregated using a joint probability formulation for non-mutually exclusive events: $F_{AI} = P_{cal} + B - (P_{cal} \times B)$.

The primary threat score is computed as:

$$\mathcal{S} = 100 \times \min\left(1, w_R R + w_{AI} F_{AI} + w_G G + w_D D\right)$$

Override: (confirmed malicious hash) $\Rightarrow \mathcal{S} = 100, \mathcal{C} = 1.0$

4.6.1 Detector Agreement & Confidence ($\mathcal{C}$)

While $\mathcal{S}$ determines the severity band, ML $\leftrightarrow$ GenAI *agreement* drives the confidence score $\mathcal{C}$. Borrowing Sysdig’s discipline of exposing confidence alongside the verdict, we define $\mathcal{C}$ as a function of the absolute difference between the two AI signals, scaled by an evidence completeness factor γ (e.g., did the dynamic sandbox successfully detonate the app?):

$$\mathcal{C} = \gamma \times (1 - |P_{cal} - B|)$$

A high threat score with low confidence (e.g., $\mathcal{S} = 90, \mathcal{C} = 0.4$, caused by a sample that refused to run dynamically but flagged heavily on static ML) is surfaced honestly rather than hidden.

After $\mathcal{S}$ and $\mathcal{C}$ are computed, the GenAI reasoning core performs a meta-check. If it flags severe qualitative disagreement (e.g., “the fused score is 30, but I observed a fake SBI overlay capturing PINs”), this disagreement does not silently alter $\mathcal{S}$. Instead, it explicitly lowers $\mathcal{C}$ and logs an analyst review flag in the evidence ledger.

4.6.2 Calibration is Mandatory

A raw classifier output is not a probability. P_{cal} is produced by isotonic (or Platt) calibration so that “0.80” genuinely means “80% of samples scored here were malicious.” Without calibration, the probabilistic fusion in F_{AI} fails mathematically. This is the rigour separating DRISHTI from a prompt simply asking an LLM to rate an app from 0 to 100.

4.6.3 Severity Bands & Automated Actions

Table 5. Severity bands and automated actions

$\mathcal{S}$	Band	Action
85–100	CRITICAL	Block; push IOCs; notify customers
65–84	HIGH	Quarantine; fast-track analyst
40–64	MEDIUM	Analyst review; monitor indicators
0–39	LOW	Log for baseline and correlation

All consequential actions in the table above (blocking, customer notification, IOC push) require explicit human analyst confirmation; DRISHTI is decision-support, not autonomous enforcement (Section 11).

4.7 M7 - Automated Reporting & Threat Intelligence

Each APK produces: a GenAI executive summary; a technical findings section with artefact hyperlinks; a MITRE ATT&CK for Mobile mapping; extracted IOCs; family attribution (or *unknown/novel*); and prioritised recommendations. DRISHTI also emits **campaign-level** intelligence:

- Auto-generated YARA rules and the Frida scripts that exposed the behaviour - so a SOC can immediately hunt for variants.
- STIX/TAXII export for SIEMs, firewalls, and channel filters.
- Campaign clustering via DEX-level fuzzy hash, shared signing certificates, and shared C2 domain.

5. The Evidence Ledger

The evidence ledger is DRISHTI’s architectural spine: an append-only, cryptographically signed directed graph where each node carries the schema:

$$n = \{\text{id, type, source_tool, content, location, confidence, timestamp}\}$$

Agents may only append; they cannot modify each other’s evidence. Every downstream artefact - each GenAI sentence, each score factor, each MITRE tag - references at least one node ID.

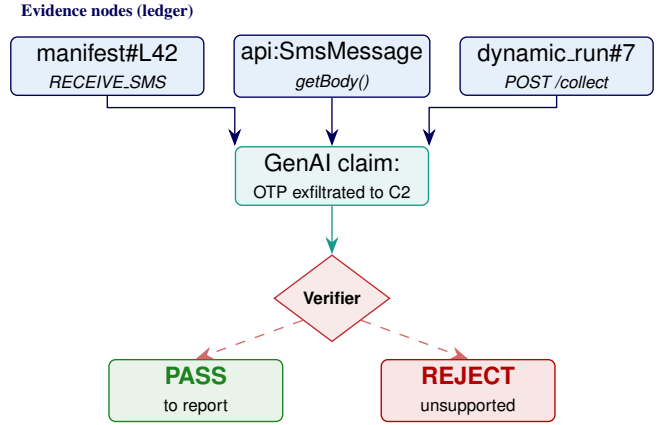


Figure 4. Evidence ledger flow: every GenAI claim must cite ledger nodes; the Verifier rejects any claim that cannot be grounded in a concrete artefact.

This delivers three properties simultaneously:

- **Anti-hallucination:** The Verifier mechanically checks that cited nodes exist and support the claim; unsupported assertions are rejected before reaching a report.
- **Explainability:** Every finding in the dashboard hyperlinks to its evidence node, enabling a one-click audit trail.
- **Reproducibility:** Re-running the same APK yields a structurally identical ledger, satisfying compliance requirements.

The design converts “an LLM said it’s malware” into “here is line-by-line proof that a compliance team can defend in court.”

DRISHTI delivers an initial Static + ML triage verdict in under 5 minutes, while the deep GenAI reverse-engineering and dynamic sandbox detonation run asynchronously to compile the final evidence ledger over a 15–30 minute window.

6. The Frontier: Adversarial Elicitation & State Synthesis

Standard sandboxes are fundamentally passive; they observe execution and fail entirely when environment-aware malware stalls due to dead infrastructure or missing target conditions. DRISHTI’s flagship novelty is evolving the sandbox into an **active adversarial interrogator**. By embedding Large Language and Vision-Language Models directly into the execution loop, the system dynamically synthesises the exact conditions required to force dormant malware into unrolling its payload, then reverse-engineers what that payload does and whom it targets.

6.1 Generative C2 Emulation (Network Reverse-Baiting)

A critical failure point in modern automated analysis occurs when a malicious Command and Control (C2) server is offline,

geo-fenced, or blocking the sandbox IP. Traditional sandboxes capture the failed connection and terminate, leaving secondary payloads unanalyzed.

DRISHTI counters this via **Generative C2 Emulation**. When the static engine (M2) identifies the network parsing logic (e.g., expecting a specific JSON schema or binary header for a "download next stage" command), the GenAI Core intercepts the outbound request. The model dynamically synthesises a correctly formatted response payload that satisfies the malware's expected schema. By spoofing the attacker's infrastructure in real-time, DRISHTI tricks the malware into executing its secondary functions and dropping its final payload directly into the sandbox's observable space.

6.2 Dynamic Sandbox Morphing (Environment Synthesis)

Third-generation trojans routinely query the Android PackageManager or device properties to verify the presence of a high-value target (e.g., specific banking applications, cryptocurrency wallets) before initiating an overlay attack. If the target is absent, the malware remains benign.

Instead of maintaining a massive, bloated sandbox pre-installed with thousands of apps, DRISHTI utilises **Dynamic Sandbox Morphing**. The moment the static code graph detects the malware querying for a specific package name, the orchestrator instantly injects synthetic registry entries, dummy APKs, and state variables into the Dalvik VM. The GenAI core actively reads the malware's evasion checks and morphs the sandbox state on the fly, creating a bespoke, irresistible victim environment tailored to the specific sample.

6.3 GenAI Reverse Engineering: Intent, Target & Victim

Forcing a sample to detonate is only half the value; the analyst needs to know *what* it does and *whom* it harms. Once behaviour is unrolled, the GenAI Code Interpreter reconstructs the malware's logic in plain English - tracing decrypted strings, exfiltration routines, and second-stage payloads back to concrete ledger artefacts. In parallel, the Social-Engineering Analyst ingests UI strings, OCR text, and the VLM impersonation signal to score the **psychological vector**: the language used (e.g., Hindi, Marathi), the regional institution impersonated, and the manipulation tactic (urgency, authority, fear). This bridges technical IOCs to business risk, yielding **Victim Demographic Profiling** - telling a bank's fraud desk *which specific customer segment needs an immediate awareness alert*, based on semantic understanding rather than hardcoded rules.

6.4 The Architectural Differentiator: Just-in-Time (JIT) Bespoke Synthesis

The true technical differentiator of DRISHTI is not merely the ability to spoof network traffic or device state, but the **speed and infinite malleability** with which it does so.

Legacy sandboxes rely on static, pre-configured environments (e.g., a standard Android image with a hardcoded list of 10 dummy contacts). If a targeted trojan expects a specific European banking application and a localised German SMS history, a traditional sandbox fails silently. Reconfiguring a legacy sandbox to catch this requires hours of manual engineering and a VM rebuild.

DRISHTI introduces **Just-in-Time (JIT) Bespoke Synthesis**. Because the GenAI Core (M4) bridges the gap between static hypothesis and dynamic execution, it designs and provisions a custom-tailored victim situation in a matter of seconds. Every evasive malware sample encounters a bespoke sandbox semantically optimised to trigger its specific payload. By using GenAI to instantly synthesise the exact environment the malware demands, DRISHTI transforms environment-awareness from an attacker's greatest advantage into a predictable, exploitable vulnerability.

7. MITRE ATT&CK for Mobile Mapping

Table 6. MITRE ATT&CK for Mobile techniques detected by DRISHTI

ID	Technique	Detection Layer
T1417	Input Capture	Accessibility hooks, screen-capture APIs (dynamic)
T1516	Input Injection	Automated clicks via Accessibility (dynamic)
T1582	SMS Control	SMS receiver + read/forward APIs (static + dynamic)
T1437	App Layer Protocol	Captured beaconing (dynamic)
T1521	Encrypted Channel	TLS + custom-crypto via cipher-API hooks
T1407	Download New Code	Dynamic class loading / second-stage fetch
T1626	Abuse Elevation	Device-admin request + anti-uninstall
T1409	Access App Data	File / credential access patterns (dynamic)
T1414	Clipboard Data	Clipboard listener registered (static)
T1641.001	Data Manipulation	Clipboard crypto-address swap

8. Defeating Evasion & Adversarial Attacks

Table 7. Adversarial evasion countermeasures

Attacker Tactic	DRISHTI Countermeasure
Obfuscation / packing	Entropy + packer detection; memory-dump unpacking; behaviour post-decryption
Emulator detection	Realistic fingerprints, fake history, sensor/GPS spoofing, physical device tier
Anti-Frida	Frida Gadget (no server); hook detection checks return benign values
Reflection / DCL	Hybrid call graph from Frida traces; dynamic edges fill static gaps
Logic / time bombs	Time-acceleration, multi-stimulus playback, extended observation windows
Dead / geo-fenced C2	Generative C2 emulation synthesises valid responses to force payload drop
Environment-aware stalling	JIT sandbox morphing injects the exact target apps and state the sample checks for
Pinned TLS / custom crypto	Frida SSL-unpinning + Cipher.doFinal hooks for pre-encryption plaintext
Adversarial ML evasion	Diverse-model ensemble; adversarial training; behaviour features hard to spoof
Prompt injection	Extracted content treated as untrusted data; strict JSON schema enforcement; verifier gate
Zero-day / novel family	Anomaly escalator forces human review; behavioural GenAI detects semantic intent

Layered resilience means an attacker must defeat *all* layers simultaneously: obfuscation hides code from static analysis but not behaviour from the sandbox; emulator detection evades the sandbox but not structural anomaly detection; a dead C2 stalls a legacy sandbox but not DRISHTI’s generative emulation; custom crypto defeats the proxy but not cipher-API hooks.

9. Evaluation & Metrics

9.1 Primary Metrics

DRISHTI is evaluated in priority order:

1. **Precision (malicious)** - banks care most; blocking a legitimate app erodes customer trust catastrophically.
2. **Recall (malicious)** - ensuring fraudulent samples are not missed.
3. **PR-AUC / ROC-AUC** - robust to class imbalance in the wild.
4. **False positive rate** - tolerable bound set by the bank’s operations team.
5. **Triage latency** - static + ML first-pass in minutes; full dynamic detonation runs asynchronously (15–30+ min) while analysts receive the preliminary verdict immediately.
6. **Explanation quality** - expert rating of factual grounding and claim-to-evidence coherence.
7. **Time-split generalisation** - trained on samples before a cut-off, tested on newer families, to validate out-of-distribution generalisation.

9.2 Datasets

Table 8. Training and evaluation datasets

Dataset	Notes
AndroZoo	Millions of APKs with metadata; signed API access required
CICMalDroid 2020	Categorised malware with behaviour logs
AMD	Family-labelled samples across major trojan lineages
Drebin	Classic feature set (~2014); historical benchmark only
MalRadard / VirusShare	Recent real-world samples; VirusShare requires invite
Benign baseline	Top legitimate apps from Google Play (financial, utility, productivity)

9.3 Projected Performance Targets

Table 9. DRISHTI performance targets vs. industry baseline

Metric	Standard Triage Baseline	DRISHTI Target
Precision (malicious)	~85%	>95%
Recall (malicious)	~80%	>90%
Level 2 threat detection (Evasive)	~25%	>80%
Mean Time to Initial Verdict (Triage)	5–10 min	<5 min
Deep Analysis & Interrogation Time	3–8 hours (Manual)	~15–30 min (GenAI)
Analyst Throughput (Samples/-Day)	2–4/day	>20/day

Baseline reflects typical static + basic sandbox automated triage. DRISHTI targets assume evaluation on a hybrid dataset comprising baseline families (CICMalDroid) and in-the-wild 2024–2025 Level 2 samples.

10. Technology Stack & Roadmap

10.1 Technology Stack

Table 10. Technology stack

Layer	Tooling
Static analysis	MobSF, Androguard, jadx, dex2jar, APKTool, Quark-Engine
Dynamic sandbox	Android Emulator, Frida + Gadget, mitmproxy, strace/eBPF, physical device farm
GenAI & RAG	Claude / open-weight code LLM; LangChain/LlamaIndex; FAISS/Chroma/pgvector
Vision-Language Model	Multimodal LLM (Gemini-/GPT-class) for UI impersonation & overlay similarity
ML	scikit-learn, XGBoost/LightGBM, PyTorch, PyTorch Geometric (GNN), SHAP
Backend	FastAPI, Celery + Redis (async jobs), Docker, PostgreSQL
Frontend & Intel	React/Next.js dashboard; STIX2/TAXII; OpenCTI-style intel store
Training data	AndroZoo, CICMalDroid, AMD, MalwareBazaar

10.2 Phased Development Roadmap

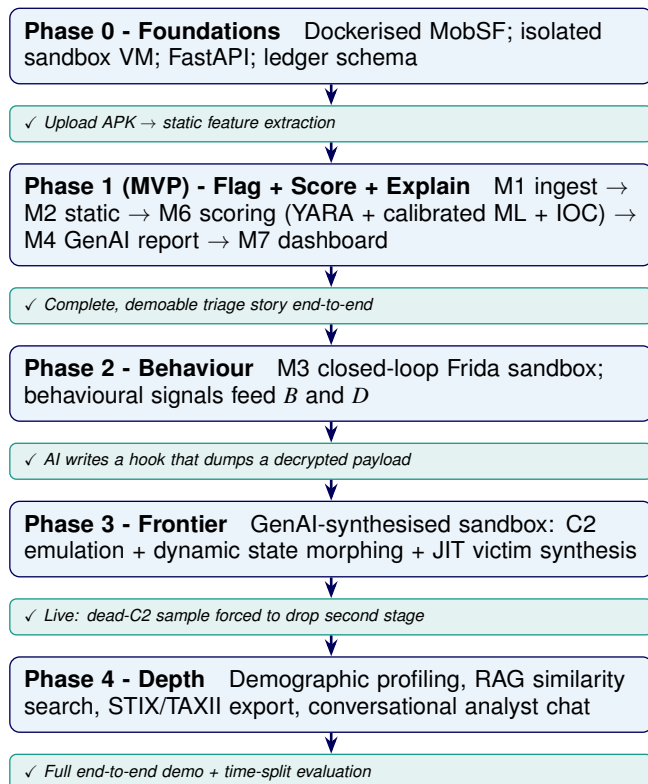


Figure 5. Phased development roadmap. Blue: phase objectives. Teal: milestone deliverables.

11. Security, Isolation & Responsible Use

Non-negotiable: never detonate APKs unsafely

The dynamic sandbox runs in a **disposable VM with no network path back to the host**, with snapshot-and-restore between samples and outbound traffic sinked or faked, so live malware cannot reach a real C2. Decoy credentials and synthetic victim data are fabricated and confined to the sandbox.

DRISHTI is a **defensive** system: its purpose is detection, explanation, scoring, and customer protection for banks and CERT teams. Research datasets are used within their licensing terms. The adversarial-elicitation modules (C2 emulation, sandbox morphing, JIT synthesis) exist solely to make evasive malware reveal behaviour inside containment; they generate no distributable malicious artefacts. Human confirmation is required before any consequential action - blocking a URL, notifying customers, or pushing IOCs to live channel filters. DRISHTI is decision-support, not autonomous enforcement.

12. Business Impact

A fraud desk analyst currently spends 3–8 hours reverse-engineering a single suspicious APK. DRISHTI compresses that deep reverse-engineering to as little as 15 minutes and surfaces an initial triage verdict in minutes, with full dynamic detonation completing asynchronously. Primary business metrics:

- **Mean time to verdict** - target: an initial static + ML verdict in minutes, with the deep GenAI interrogation completing asynchronously in 15–30 minutes (versus 3–8 hours of manual reverse-engineering).
- **Analyst throughput multiplier** - target: 10× increase in samples reviewed per analyst per day.
- **Campaign neutralisation time** - from sample receipt to IOC distribution via TAXII, enabling proactive blocking of the full variant family within the same working session.
- **Customer impact** - demographic profiling enables a bank to send targeted SMS/app warnings to the at-risk segment *before* fraud is attempted, not after.

The auto-generated YARA rules and Frida scripts are the highest-leverage output: they allow a bank's SOC to immediately hunt for all variants sharing the detected behaviour across its entire monitored device fleet, not just the submitted sample. A single DRISHTI verdict on one sample can proactively protect against an entire campaign.

13. Conclusion

DRISHTI addresses the full spectrum of APK-borne financial fraud: Level 1 malicious artefacts detectable by static and ML analysis, and the harder Level 2 latent threats that evade every existing scanner. Its core innovation is a GenAI reasoning layer that does not merely classify - it acts as an automated reverse engineer, synthesising the very environment a dormant trojan is hunting for to make it confess, then binding every sentence of its verdict to a concrete artefact that an analyst can verify.

The evidence ledger transforms AI output from an opaque score into a machine-generated audit trail. The GenAI-synthesised sandbox - emulating dead C2 infrastructure and morphing device state Just-in-Time - unrolls behaviour that dormant trojans conceal from generic sandboxes. The multi-label ML ensemble provides signature-agnostic generic checks and calibrates confidence. GenAI reverse engineering exposes the malware's intent, impersonated target, and victim demographic. Together, these layers deliver what bank fraud desks actually need: not "this looks malicious" but "here is exactly what this app does, here is the proof, here is the YARA rule to find its siblings, and here is who it targets."

The DRISHTI thesis

GenAI is a first-class analyst inside the detection pipeline - reading decompiled code and live behaviour the way a human reverse engineer would, and actively synthesising the conditions that force malware to reveal itself - while the threat score is produced by a calibrated, explainable, multi-signal engine. Every verdict is bound to cryptographic evidence, and a human confirms every consequential action. The AI reasons; it never guesses the number.

Every other system classifies an APK. **DRISHTI interrogates it.**

References

- [1] VirusTotal. *Introducing VirusTotal Code Insight: Empowering threat analysis with generative AI*. <https://blog.virustotal.com/2023/04/introducing-virustotal-code-insight.html>
- [2] VirusTotal. *Empowering Defenders: How AI is shaping malware analysis*. <https://assets.virustotal.com/reports/2023-ai>
- [3] Sysdig. *More than an Assistant - A New Architecture for GenAI in Cloud Security*. <https://www.sysdig.com/blog/genai-cloud-security>
- [4] Sysdig. *Sysdig Sage: A groundbreaking AI security analyst*. <https://www.sysdig.com/blog/sysdig-sage-a-groundbreaking-ai-security-analyst>
- [5] Sysdig. *What is Falco? Open source runtime threat detection*. <https://www.sysdig.com/learn-cloud-native/what-is-falco>
- [6] Reserve Bank of India. *Annual Report on Cyber Security in the Indian Banking Sector, 2024*. <https://www.rbi.org.in>
- [7] MITRE. *ATT&CK for Mobile*. <https://attack.mitre.org/matrices/mobile/>
- [8] Mobile Security Framework (MobSF). <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [9] Frida - Dynamic instrumentation toolkit. <https://frida.re/>
- [10] Androguard. <https://github.com/androguard/androguard>
- [11] YARA. <https://virustotal.github.io/yara/>
- [12] abuse.ch - MalwareBazaar & URLhaus. <https://abuse.ch/>
- [13] AndroZoo dataset. <https://androzoo.uni.lu/>
- [14] CIC-MalDroid 2020. <https://www.unb.ca/cic/datasets/maldroid-2020.html>
- [15] Quark-Engine. <https://github.com/quark-project/quark-engine>
- [16] Github Report Link. <https://github.com/AyushaH2005/DRISHTI-Dynamic-Reasoning-Interrogation-System-for-Heuristic-Threat-Identification/blob/main/README.md>